

Universidad de Costa Rica

Sede Regional del Pacífico

Bachillerato en Informática Empresarial

Desarrollo de Software IV

Proyecto integrador

Documentación técnica

BecaGest

Sistema de gestión de becas estudiantiles

Integrantes:

Ana Fabiola Jiménez Montero C19112

Maria Fernanda Martínez Bello C34593

Valentina Badilla Morera C20913

Docente:

Irwin Leal

Fecha de entrega:

29 de octubre de 2026

BecaGest: Sistema de gestión de becas estudiantiles

La transformación digital ha permitido desarrollar soluciones tecnológicas orientadas a mejorar la administración de información y la ejecución de procesos en diferentes ámbitos institucionales. En el sector educativo, la gestión de becas estudiantiles constituye una actividad importante, debido a que requiere organizar información personal, administrar convocatorias, evaluar solicitudes y controlar la asignación de beneficios económicos.

En este contexto, surge BecaGest, un sistema desarrollado como una interfaz de programación de aplicaciones de tipo REST (API REST), cuyo propósito es facilitar la gestión de becas estudiantiles mediante un proceso organizado y centralizado. El proyecto integra diferentes módulos funcionales que permiten administrar la información y establecer relaciones entre estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones.

Para su desarrollo se utilizaron Python, Django y Django REST Framework, herramientas que permiten construir servicios backend y gestionar operaciones sobre los datos. Asimismo, se implementó MySQL como sistema de gestión de bases de datos, con el objetivo de almacenar la información de manera estructurada y mantener las relaciones entre las entidades del sistema.

BecaGest también incorpora mecanismos de autenticación mediante tokens y controles de autorización basados en roles, con el propósito de restringir determinadas operaciones según las responsabilidades de cada usuario. Adicionalmente, se implementaron reglas de negocio, consultas mediante Django ORM y pruebas que permiten verificar el funcionamiento de los diferentes módulos.

La validación del sistema se realiza mediante pruebas automatizadas de Django y solicitudes HTTP ejecutadas desde Postman. Estas verificaciones permiten evaluar el comportamiento de los endpoints, los permisos, las restricciones y las respuestas generadas ante diferentes situaciones.

Por lo tanto, el desarrollo de BecaGest representa una aplicación práctica de los conocimientos adquiridos en el área de desarrollo de software, mediante la integración de tecnologías de programación, bases de datos, servicios REST y mecanismos de seguridad.

Descripción del proyecto

BecaGest es una API REST orientada a la administración de becas estudiantiles. El sistema permite gestionar las distintas etapas del proceso, desde el registro de estudiantes hasta la asignación de beneficios económicos, utilizando una estructura de cinco módulos relacionados.

El módulo de estudiantes administra la información de las personas registradas. El módulo de convocatorias permite establecer los períodos de participación y los cupos disponibles. Por su parte, el módulo de solicitudes registra las peticiones realizadas por los estudiantes, mientras que el módulo de evaluaciones permite determinar si estas cumplen con los criterios correspondientes. Finalmente, el módulo de asignaciones administra los beneficios económicos otorgados a las solicitudes aprobadas.

El sistema cuenta con tres perfiles de usuario: administrador, encargado y estudiante. Cada perfil dispone de permisos específicos para acceder a determinadas funcionalidades, permitiendo establecer restricciones sobre la consulta y modificación de información.

La persistencia de los datos se realiza mediante MySQL, utilizando Django ORM para interactuar con las tablas y sus relaciones. Las funcionalidades se encuentran disponibles a través de endpoints HTTP que pueden ser utilizados y comprobados mediante Postman.

Objetivo general

Desarrollar una API REST para la gestión de becas estudiantiles mediante Django, Django REST Framework y MySQL, que permita administrar estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones, incorporando mecanismos de autenticación, autorización y validación de datos para garantizar el funcionamiento organizado del sistema.

Objetivos específicos

1. Diseñar un modelo de datos relacional que permita almacenar y relacionar la información correspondiente a los estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones de becas.
2. Implementar cinco módulos funcionales mediante Django y Django REST Framework, utilizando endpoints que permitan realizar operaciones de creación, consulta, actualización y eliminación de información según las reglas establecidas.
3. Configurar una base de datos MySQL que permita almacenar los registros del sistema y mantener la integridad de las relaciones mediante claves primarias, claves foráneas y restricciones.

4. Establecer mecanismos de autenticación mediante tokens y controles de autorización basados en los roles de administrador, encargado y estudiante.
5. Incorporar reglas de negocio que permitan validar la información registrada y controlar situaciones como solicitudes duplicadas, fechas incorrectas, puntajes inválidos y asignaciones que no cumplan las condiciones requeridas.
6. Implementar consultas mediante Django ORM que permitan recuperar información relacionada, aplicar filtros y generar reportes sobre el funcionamiento del sistema.
7. Verificar las funcionalidades desarrolladas mediante pruebas automatizadas y solicitudes HTTP realizadas desde Postman, comprobando el comportamiento de los endpoints y las respuestas del sistema.

Metodología de desarrollo

El desarrollo de BecaGest se estructuró mediante un proceso organizado en etapas, orientado a la construcción de una API REST que permitiera integrar las diferentes funcionalidades relacionadas con la gestión de becas estudiantiles. Este procedimiento comprendió la identificación de los requerimientos, el diseño del modelo de datos, la implementación de los módulos, la configuración de los mecanismos de seguridad y la ejecución de pruebas funcionales.

Durante la etapa inicial se establecieron los componentes necesarios para representar el proceso de gestión de becas. A partir de esta definición, se identificaron cinco módulos funcionales: estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones. La organización de estos componentes permitió establecer las responsabilidades de cada módulo y determinar las relaciones necesarias para el intercambio de información.

Posteriormente, se diseñó el modelo de datos utilizando las herramientas proporcionadas por Django ORM. Se definieron las entidades, sus atributos y las relaciones correspondientes, incluyendo claves primarias, claves foráneas y restricciones destinadas a preservar la integridad de los registros. La persistencia de la información se configuró mediante MySQL, mientras que las migraciones de Django permitieron establecer la estructura de las tablas.

La siguiente etapa comprendió la implementación de las funcionalidades del sistema mediante Django REST Framework. Para ello, se desarrollaron endpoints HTTP destinados a ejecutar operaciones de creación, consulta, actualización y eliminación, considerando las características y restricciones propias de cada recurso.

Asimismo, se incorporaron mecanismos de autenticación mediante tokens y controles de autorización basados en los perfiles de administrador, encargado y estudiante. Estos mecanismos permitieron establecer diferencias entre las operaciones disponibles para cada tipo de usuario y restringir el acceso a determinadas funcionalidades.

Finalmente, se desarrolló un proceso de verificación mediante pruebas automatizadas de Django y pruebas manuales realizadas desde Postman. Estas comprobaciones permitieron evaluar el funcionamiento de los módulos, las reglas de negocio, las restricciones de acceso y las respuestas HTTP generadas por el sistema.

Arquitectura del sistema

BecaGest presenta una arquitectura backend basada en Django y Django REST Framework. Su estructura permite organizar las responsabilidades de procesamiento, validación y persistencia de información, facilitando la comunicación entre los diferentes módulos del sistema.

La interacción con la aplicación se realiza mediante solicitudes HTTP enviadas a los endpoints disponibles. Estas solicitudes son procesadas por las vistas correspondientes, las cuales ejecutan las operaciones necesarias según el recurso solicitado y las condiciones de autorización establecidas.

Los serializadores permiten transformar los datos entre las representaciones utilizadas por la API y los objetos administrados por Django. Además, incorporan validaciones destinadas a comprobar que la información recibida cumpla con los requisitos definidos para cada entidad.

La comunicación con MySQL se realiza mediante Django ORM, el cual permite consultar, registrar, actualizar y eliminar información utilizando los modelos del sistema. Este mecanismo facilita el manejo de relaciones entre las entidades y la aplicación de consultas que involucran diferentes tablas.

En cuanto a la seguridad, la arquitectura incorpora autenticación basada en tokens y controles de acceso según los grupos definidos en Django. De esta manera, las funcionalidades disponibles se encuentran condicionadas por las responsabilidades y permisos asociados a cada usuario.

Organización modular

La estructura funcional de BecaGest se encuentra distribuida en cinco aplicaciones de Django, cada una encargada de administrar un componente específico del proceso de gestión de becas.

El módulo de estudiantes permite administrar la información académica y personal requerida para identificar a quienes participan en las convocatorias.

El módulo de convocatorias gestiona las oportunidades de acceso a beneficios económicos, considerando los períodos de inscripción, los cupos y el estado de disponibilidad.

El módulo de solicitudes permite relacionar a los estudiantes con las convocatorias en las que desean participar, manteniendo un registro del motivo y del estado de cada solicitud.

El módulo de evaluaciones registra los resultados del proceso de revisión de solicitudes, incorporando puntajes, observaciones y decisiones de aprobación o rechazo.

Finalmente, el módulo de asignaciones administra los beneficios económicos derivados de evaluaciones aprobadas, estableciendo los montos mensuales y el estado correspondiente.

Esta distribución funcional permite mantener una separación de responsabilidades entre las aplicaciones, sin perder la integración necesaria para ejecutar el proceso general de gestión de becas estudiantiles.

Diseño de la base de datos

El diseño de la base de datos de BecaGest se fundamenta en un modelo relacional que permite organizar, almacenar y gestionar la información correspondiente al proceso de administración de becas estudiantiles. Su estructura establece relaciones entre las entidades del sistema, facilitando el intercambio de información y contribuyendo a mantener la integridad de los registros.

Para la implementación de la base de datos se utiliza MySQL, mientras que la definición de las entidades y sus relaciones se realiza mediante Django ORM. Esta herramienta permite representar las tablas como modelos de Python, administrar sus atributos y establecer las relaciones correspondientes mediante claves primarias y foráneas.

La base de datos, denominada becagest_db, contiene las tablas asociadas a los cinco módulos principales: estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones. Adicionalmente, se utilizan las tablas proporcionadas por el sistema de autenticación de Django para administrar usuarios, grupos y permisos.

La organización de estas entidades permite representar el funcionamiento general del sistema, desde el registro inicial de un estudiante hasta la asignación de una beca, considerando las restricciones y condiciones establecidas durante cada etapa.

Descripción de las entidades

Entidad Estudiante

La entidad Estudiante almacena la información de las personas registradas en el sistema. Cada estudiante se encuentra asociado a una cuenta de usuario de Django, lo que permite relacionar sus datos con los mecanismos de autenticación y autorización.

Los principales atributos son:

id: identificador único del estudiante.

usuario: relación con el usuario registrado en Django.

identificacion: número de identificación del estudiante.

nombre: nombre del estudiante.

apellidos: apellidos del estudiante.

carrera: carrera académica a la que pertenece.

activo: indica si el estudiante se encuentra habilitado.

El atributo identificacion presenta una restricción de unicidad para evitar registros duplicados. Asimismo, la relación con el usuario de Django permite establecer una correspondencia individual entre cada cuenta y su respectiva ficha estudiantil.

Entidad Convocatoria

La entidad Convocatoria administra la información correspondiente a los períodos de solicitud de becas. Su función consiste en establecer las condiciones generales bajo las cuales los estudiantes pueden presentar sus solicitudes.

Los principales atributos son:

id: identificador único de la convocatoria.

nombre: nombre de la convocatoria.

descripcion: información descriptiva del beneficio ofrecido.

fecha_inicio: fecha de apertura del período de inscripción.

fecha_fin: fecha de finalización del período.

cupos: cantidad de beneficios disponibles.

activa: indica si la convocatoria se encuentra habilitada.

La entidad incorpora validaciones destinadas a garantizar que la fecha de finalización no sea anterior a la fecha de inicio y que la cantidad de cupos sea mayor que cero.

Entidad Solicitud

La entidad Solicitud representa las peticiones de becas realizadas por los estudiantes. Su estructura permite vincular a cada estudiante con una convocatoria y conservar información relacionada con el motivo y el estado de su solicitud.

Los principales atributos son:

id: identificador único de la solicitud.

estudiante: clave foránea asociada a la entidad Estudiante.

convocatoria: clave foránea asociada a la entidad Convocatoria.

motivo: justificación presentada por el estudiante.

fecha_solicitud: fecha de registro de la solicitud.

estado: situación actual de la solicitud.

Los estados contemplados son PENDIENTE, APROBADA y RECHAZADA.

La entidad incorpora una restricción que impide registrar más de una solicitud del mismo estudiante para una misma convocatoria. Esta condición permite evitar duplicidades y mantener un control adecuado sobre la participación estudiantil.

Entidad Evaluación

La entidad Evaluación almacena los resultados correspondientes al proceso de revisión de solicitudes. Su propósito consiste en registrar la decisión tomada sobre cada solicitud y conservar información que permita justificar dicha decisión.

Los principales atributos son:

id: identificador único de la evaluación.

solicitud: relación individual con la solicitud evaluada.

resultado: decisión de aprobación o rechazo.

puntaje: calificación asignada durante la evaluación.

observaciones: información complementaria sobre la decisión.

fecha_evaluacion: fecha de registro de la evaluación.

El resultado puede establecerse como APROBADA o RECHAZADA. Además, el puntaje debe mantenerse dentro de un rango comprendido entre 0 y 100.

Cada solicitud puede tener como máximo una evaluación, lo que evita registrar decisiones duplicadas para una misma petición.

Entidad Asignación

La entidad Asignación administra los beneficios económicos otorgados a partir de las evaluaciones aprobadas. Esta entidad permite conservar información sobre el monto mensual asignado y el estado del beneficio.

Los principales atributos son:

id: identificador único de la asignación.

evaluacion: relación individual con la evaluación correspondiente.

monto_mensual: cantidad económica otorgada mensualmente.

fecha_asignacion: fecha de registro del beneficio.

estado: condición actual de la asignación.

Los estados contemplados son ACTIVA, FINALIZADA y CANCELADA.

El monto mensual debe ser mayor que cero. Asimismo, el registro de una asignación requiere que exista una evaluación aprobada y que la convocatoria correspondiente disponga de cupos.

Modelo entidad-relación

El modelo entidad-relación de BecaGest representa las asociaciones existentes entre las diferentes entidades del sistema y permite identificar la cardinalidad correspondiente a cada vínculo.

Relaciones entre entidades

Relación entre Usuario y Estudiante

Se establece una relación de uno a uno (1:1), debido a que cada ficha de estudiante se encuentra asociada a una cuenta de usuario específica de Django.

Relación entre Estudiante y Solicitud

Se establece una relación de uno a muchos (1). Un estudiante puede registrar diferentes solicitudes, siempre que correspondan a convocatorias distintas. Por su parte, cada solicitud pertenece a un único estudiante.

Relación entre Convocatoria y Solicitud

Se establece una relación de uno a muchos (1). Una convocatoria puede recibir solicitudes de diferentes estudiantes, mientras que cada solicitud corresponde a una convocatoria específica.

Relación entre Solicitud y Evaluación

Se establece una relación de uno a uno (1:1). Una solicitud puede disponer de una evaluación, mientras que cada evaluación corresponde a una única solicitud.

Relación entre Evaluación y Asignación

Se establece una relación de uno a uno (1:1). Una evaluación aprobada puede originar una asignación de beca, mientras que cada asignación se encuentra vinculada a una evaluación específica.

Relación entre Usuario y Grupo

El sistema de autenticación de Django establece una relación de muchos a muchos (N) entre usuarios y grupos. Esta asociación permite asignar perfiles y administrar los permisos correspondientes a cada usuario.

Integridad referencial

La integridad referencial de la base de datos se mantiene mediante la utilización de claves primarias, claves foráneas, relaciones individuales y restricciones de unicidad.

Las claves primarias permiten identificar cada registro de manera independiente, mientras que las claves foráneas establecen las conexiones entre las entidades relacionadas.

Adicionalmente, las validaciones implementadas en los módulos permiten controlar situaciones que podrían afectar la consistencia de la información, como la duplicación de solicitudes, el registro de evaluaciones repetidas y la asignación de beneficios sin cumplir con las condiciones establecidas.

Las restricciones relacionadas con la eliminación y modificación de determinados registros también contribuyen a preservar la información necesaria para mantener la continuidad del proceso de gestión de becas.

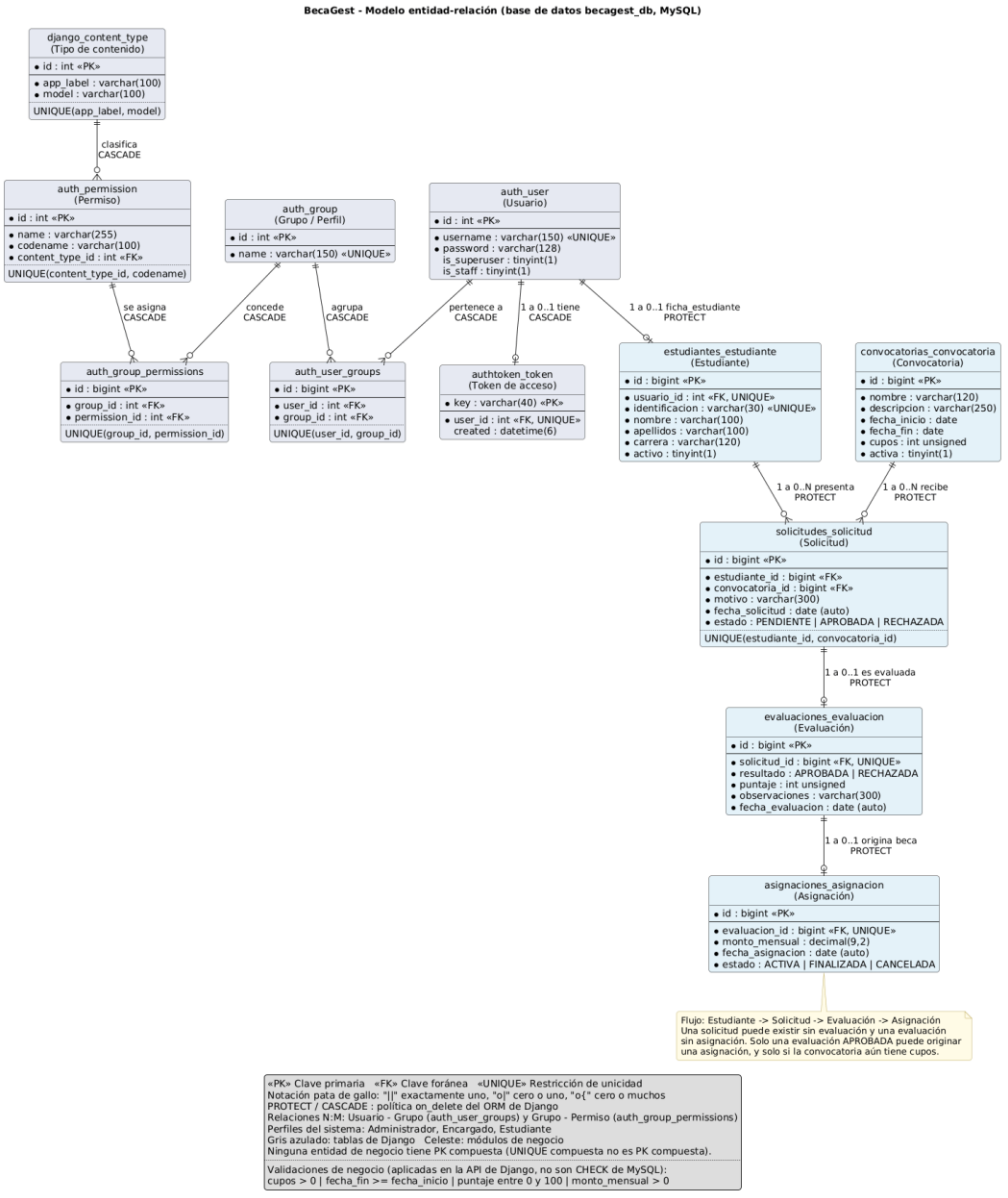


Figura 1

Diagrama UML del modelo de datos e integridad referencial del sistema BecaGest

Implementación de los módulos funcionales

La implementación de BecaGest se desarrolló mediante cinco aplicaciones de Django que permiten administrar las diferentes etapas del proceso de gestión de becas estudiantiles. Cada aplicación dispone de componentes específicos para el manejo de los modelos, la validación de información, el procesamiento de solicitudes HTTP y la definición de rutas de acceso.

La organización modular facilita la separación de responsabilidades y permite mantener una relación funcional entre los componentes. De esta manera, la información registrada en una etapa puede utilizarse posteriormente en los procesos de solicitud, evaluación y asignación de beneficios económicos.

Módulo de estudiantes

El módulo de estudiantes permite administrar la información correspondiente a las personas registradas en el sistema. Su implementación se fundamenta en un modelo de Django que almacena datos de identificación, nombres, apellidos, carrera académica y estado de actividad.

Cada registro estudiantil se encuentra vinculado con un usuario del sistema de autenticación de Django mediante una relación de uno a uno. Esta asociación permite identificar al propietario de la información y aplicar restricciones de acceso según el perfil correspondiente.

El módulo incorpora operaciones para registrar, consultar, actualizar y eliminar información, considerando los permisos establecidos para cada tipo de usuario. El administrador dispone de las funciones de gestión, mientras que los estudiantes únicamente pueden consultar su información autorizada.

Asimismo, se implementaron mecanismos de validación para evitar identificaciones duplicadas y comprobar que los datos registrados cumplan con las condiciones establecidas. Las consultas permiten aplicar filtros por carrera y estado de actividad, además de mecanismos de búsqueda y paginación.

Módulo de convocatorias

El módulo de convocatorias administra los períodos durante los cuales los estudiantes pueden presentar solicitudes para obtener beneficios económicos.

Su modelo de datos contiene información relacionada con el nombre de la convocatoria, descripción, fecha de inicio, fecha de finalización, cantidad de cupos disponibles y estado de actividad.

La implementación permite registrar nuevas convocatorias, consultar las existentes, modificar sus características y eliminar registros cuando las condiciones del sistema lo permiten.

Las operaciones de creación, modificación y eliminación se encuentran restringidas al perfil de administrador. Por otra parte, los usuarios autenticados pueden consultar las convocatorias disponibles.

Entre las validaciones implementadas se encuentra la comprobación de que la fecha de finalización no sea anterior a la fecha de inicio. También se exige que la cantidad de cupos sea mayor que cero y que las modificaciones no comprometan las asignaciones que deban conservarse.

Las consultas REST incorporan filtros por estado de actividad y período de apertura, búsqueda por nombre o descripción, ordenamiento de resultados y paginación.

Módulo de solicitudes

El módulo de solicitudes permite registrar las peticiones de becas realizadas por los estudiantes y relacionarlas con las convocatorias correspondientes.

Cada solicitud contiene una referencia al estudiante, una referencia a la convocatoria, el motivo de la petición, la fecha de registro y el estado del proceso.

Inicialmente, las solicitudes se registran con el estado PENDIENTE. Posteriormente, el resultado de la evaluación determina su actualización a APROBADA o RECHAZADA.

La implementación incorpora restricciones destinadas a evitar que un estudiante presente varias solicitudes para una misma convocatoria. Esta condición se complementa con la validación del período de inscripción, el estado de actividad de la convocatoria y la situación del estudiante.

El acceso a las solicitudes se encuentra condicionado por los permisos del usuario autenticado. Los estudiantes pueden registrar y consultar sus propias solicitudes, mientras que el personal autorizado dispone de funciones adicionales de administración.

También se establecieron restricciones para impedir la modificación de solicitudes que ya fueron evaluadas, contribuyendo a preservar la consistencia de la información.

Módulo de evaluaciones

El módulo de evaluaciones administra la revisión y valoración de las solicitudes presentadas por los estudiantes.

Cada evaluación se encuentra relacionada con una solicitud específica mediante una asociación de uno a uno. Su modelo contiene el resultado de la evaluación, el puntaje asignado, las observaciones correspondientes y la fecha de registro.

La creación de evaluaciones está reservada a los usuarios con permisos de administrador o encargado. Esta restricción evita que los estudiantes puedan modificar directamente las decisiones del proceso.

El sistema establece un rango válido de puntajes comprendido entre 0 y 100. Asimismo, impide registrar más de una evaluación para una misma solicitud y restringe la evaluación de solicitudes que no se encuentren pendientes.

Cuando se registra una evaluación, el estado de la solicitud relacionada se actualiza de acuerdo con el resultado obtenido. Esta operación permite mantener la correspondencia entre las decisiones registradas y la situación de cada solicitud.

Adicionalmente, se implementaron restricciones que impiden modificar una evaluación cuando ya existe una asignación de beca relacionada.

Módulo de asignaciones

El módulo de asignaciones administra los beneficios económicos otorgados como resultado de las evaluaciones aprobadas.

Cada asignación se vincula con una evaluación específica mediante una relación de uno a uno. Los datos registrados incluyen el monto mensual del beneficio, la fecha de asignación y el estado correspondiente.

El sistema contempla los estados ACTIVA, FINALIZADA y CANCELADA, los cuales permiten identificar la situación del beneficio económico.

Las operaciones de asignación están restringidas a los perfiles autorizados. Además, se establecieron validaciones para comprobar que la evaluación relacionada se encuentre aprobada y que existan cupos disponibles en la convocatoria correspondiente.

El monto mensual debe ser mayor que cero, evitando el registro de asignaciones con cantidades económicas inválidas.

Este módulo constituye la etapa final del proceso de gestión de becas y permite relacionar los resultados de las evaluaciones con los beneficios económicos otorgados.

Integración entre módulos

La integración funcional de BecaGest se fundamenta en las relaciones establecidas entre las cinco aplicaciones del sistema.

El proceso inicia con el registro de estudiantes y la creación de convocatorias. Posteriormente, los estudiantes presentan solicitudes para participar en las convocatorias disponibles.

Las solicitudes registradas son revisadas por el personal autorizado mediante el módulo de evaluaciones. Una vez determinada la aprobación de una solicitud, el sistema permite registrar la asignación del beneficio correspondiente, siempre que se cumplan las condiciones establecidas.

Esta estructura permite desarrollar un proceso continuo, en el cual las operaciones realizadas sobre una entidad mantienen relación con la información registrada en las demás.

La utilización de claves foráneas, relaciones individuales, restricciones de unicidad y mecanismos de validación contribuye a mantener la integridad y coherencia de los registros durante las diferentes etapas del proceso.

Autenticación y autorización del sistema

La seguridad de BecaGest se fundamenta en mecanismos de autenticación y autorización que permiten controlar el acceso a las funcionalidades del sistema. Estos mecanismos tienen como finalidad identificar a los usuarios, verificar sus permisos y restringir aquellas operaciones que no corresponden a sus responsabilidades.

La implementación utiliza el sistema de autenticación de Django y las funcionalidades proporcionadas por Django REST Framework. Mediante estas herramientas se establece un control de acceso basado en tokens y grupos de usuarios, lo que permite diferenciar las operaciones disponibles para cada perfil.

La autenticación permite comprobar la identidad de un usuario, mientras que la autorización determina las acciones que puede ejecutar sobre los recursos disponibles. La combinación de

ambos mecanismos contribuye a proteger la información registrada y a mantener un funcionamiento controlado de las operaciones relacionadas con las becas estudiantiles.

Autenticación mediante tokens

BecaGest utiliza autenticación mediante tokens para identificar a los usuarios que realizan solicitudes a la API REST. Este mecanismo permite asociar cada petición autenticada con una cuenta registrada en Django.

El proceso inicia cuando un usuario proporciona sus credenciales a través del endpoint correspondiente. Si los datos son válidos, el sistema devuelve un token de autenticación que puede utilizarse posteriormente para acceder a los recursos protegidos.

El endpoint utilizado es:

POST /api/token/

El cuerpo de la solicitud contiene el nombre de usuario y la contraseña. Una vez obtenido el token, este se incorpora al encabezado Authorization de las peticiones HTTP.

El formato utilizado es:

Authorization: Token VALOR_DEL_TOKEN

Cuando una solicitud contiene credenciales válidas, Django REST Framework identifica al usuario y permite continuar con la verificación de los permisos correspondientes.

Por el contrario, cuando se intenta acceder a un recurso protegido sin proporcionar credenciales, el sistema responde con el código HTTP 401 Unauthorized.

Este comportamiento permite impedir el acceso anónimo a las funcionalidades que requieren autenticación.

Autorización basada en roles

La autorización en BecaGest se organiza mediante tres grupos de usuarios: Administrador, Encargado y Estudiante.

Cada grupo representa un conjunto de responsabilidades relacionadas con las actividades de gestión de becas. Los controles de autorización permiten determinar las operaciones disponibles según el perfil del usuario autenticado.

Administrador

El administrador dispone de permisos para gestionar la información general del sistema.

Entre sus responsabilidades se encuentran el registro y mantenimiento de estudiantes, la administración de convocatorias y la supervisión de los procesos relacionados con las solicitudes, evaluaciones y asignaciones.

Asimismo, puede consultar los reportes administrativos generados por el sistema.

Encargado

El encargado participa en las actividades operativas relacionadas con la revisión y administración de las solicitudes de becas.

Este perfil puede consultar información de estudiantes y convocatorias, gestionar solicitudes, registrar evaluaciones y administrar asignaciones de beneficios económicos.

También dispone de acceso a los reportes administrativos, lo que permite dar seguimiento al estado de las solicitudes y a los beneficios registrados.

Sin embargo, no cuenta con autorización para crear, modificar o eliminar directamente los registros de estudiantes y convocatorias.

Estudiante

El estudiante dispone de permisos limitados a las operaciones correspondientes a su participación en el proceso de solicitud de becas.

Este perfil puede consultar la información permitida, visualizar las convocatorias y registrar solicitudes vinculadas con su propia cuenta.

Además, puede consultar sus solicitudes y los resultados de las evaluaciones o asignaciones relacionadas con estas.

Las modificaciones y eliminaciones de solicitudes propias únicamente se permiten cuando estas se encuentran pendientes y cumplen con las restricciones establecidas.

El estudiante no tiene autorización para registrar evaluaciones, asignar beneficios económicos ni consultar reportes administrativos.

Matriz de permisos

La matriz de permisos permite identificar las operaciones autorizadas para cada perfil de usuario. Su finalidad consiste en documentar las restricciones de acceso establecidas en los diferentes módulos y recursos de BecaGest.

Tabla 1

Matriz de permisos de los usuarios del sistema BecaGest

Operación	**Administrador**	**Encargado**	**Estudiante**
Consultar estudiantes	Sí	Sí	Solo propio
Crear, modificar y eliminar estudiantes	Sí	No	No
Consultar convocatorias	Sí	Sí	Sí
Crear, modificar y eliminar convocatorias	Sí	No	No
Consultar solicitudes	Todas	Todas	Solo propias
Registrar solicitudes	Sí	Sí	Solo propia
Modificar o eliminar solicitudes pendientes	Sí	Sí	Solo propias
Consultar evaluaciones	Todas	Todas	Solo propias
Registrar y gestionar evaluaciones	Sí	Sí	No
Consultar asignaciones	Todas	Todas	Solo propias
Registrar y gestionar asignaciones	Sí	Sí	No
Registrar y gestionar asignaciones	Sí	Sí	No
Consultar reportes administrativos	Sí	Sí	No
Consultar perfil autenticado	Sí	Sí	Sí

Nota. Elaboración propia a partir de los mecanismos de autenticación y autorización implementados en BecaGest. Los permisos establecidos para cada perfil se encuentran sujetos a las reglas de negocio y restricciones aplicables a las operaciones del sistema.

Control de acceso a los recursos

El control de acceso a los recursos se establece mediante la comprobación de la identidad del usuario y de los permisos correspondientes a cada operación.

En aquellos módulos que contienen información asociada directamente con los estudiantes, las consultas se restringen para evitar que un usuario acceda a registros que pertenecen a otra cuenta.

Asimismo, las operaciones administrativas se encuentran reservadas para los perfiles autorizados, de acuerdo con las responsabilidades definidas en la matriz de permisos.

La aplicación de estos controles permite diferenciar entre usuarios autenticados que poseen autorización para ejecutar determinadas acciones y usuarios que, aun contando con credenciales válidas, no disponen de los permisos necesarios.

Verificación de seguridad

La seguridad del sistema se comprobó mediante solicitudes HTTP realizadas desde Postman, utilizando usuarios con diferentes perfiles y condiciones de autenticación.

Entre las verificaciones efectuadas se encuentra el intento de acceso al endpoint de perfil sin proporcionar un token. Como resultado, el sistema devolvió el código HTTP 401 Unauthorized, indicando que no se habían proporcionado las credenciales necesarias.

También se comprobó el comportamiento de las restricciones de autorización mediante el intento de registrar una evaluación utilizando una cuenta de estudiante. En este caso, el sistema respondió con el código HTTP 403 Forbidden, debido a que el usuario no disponía del permiso requerido.

De manera similar, se verificó que una cuenta con el perfil Estudiante no pudiera consultar el resumen administrativo del sistema. La respuesta obtenida fue 403 Forbidden, lo que confirmó la aplicación de la restricción correspondiente.

Posteriormente, se realizó la misma consulta utilizando una cuenta con el perfil Encargado. El sistema respondió con el código HTTP 200 OK y presentó los datos del resumen administrativo, demostrando la diferencia de permisos entre ambos perfiles.

Estas verificaciones evidencian el funcionamiento de los mecanismos de autenticación y autorización implementados en BecaGest, así como la aplicación de restricciones de acceso según las responsabilidades asignadas a cada usuario.

Validaciones y reglas de negocio

La implementación de reglas de negocio constituye un componente fundamental en el funcionamiento de BecaGest, debido a que permite establecer condiciones para el registro, modificación y procesamiento de la información correspondiente a las becas estudiantiles. Estas reglas tienen como finalidad garantizar la coherencia de los datos, evitar operaciones incompatibles con los procedimientos establecidos y mantener la integridad de los registros durante las diferentes etapas del sistema.

Las validaciones se implementaron mediante los modelos, serializadores y vistas de Django, utilizando las funcionalidades proporcionadas por Django REST Framework. Esta estructura permite comprobar la información enviada por los usuarios antes de ejecutar determinadas operaciones sobre la base de datos.

Asimismo, las restricciones se aplican tanto a los procesos de creación como a las operaciones de actualización, incluyendo aquellas realizadas mediante el método HTTP PATCH. De esta manera, se procura evitar que una modificación parcial permita registrar información contraria a las condiciones del sistema.

Reglas de negocio implementadas

Regla 1. Identificación única de estudiantes

Cada estudiante debe contar con un número de identificación único. Esta restricción impide el registro de estudiantes con identificaciones duplicadas y contribuye a mantener la integridad de la información personal.

Regla 2. Validación de fechas de convocatorias

La fecha de finalización de una convocatoria debe ser igual o posterior a su fecha de inicio. El sistema rechaza las convocatorias que presentan períodos cronológicamente incorrectos.

Regla 3. Validación de cupos disponibles

Toda convocatoria debe contar con una cantidad de cupos mayor que cero. Esta condición permite evitar la creación de convocatorias que no dispongan de capacidad para otorgar beneficios económicos.

Regla 4. Restricción de solicitudes duplicadas

Un estudiante únicamente puede presentar una solicitud para una misma convocatoria. Esta regla se complementa con una restricción de unicidad compuesta sobre los campos estudiante y convocatoria.

Regla 5. Disponibilidad de las convocatorias

Las solicitudes únicamente pueden registrarse cuando la convocatoria correspondiente se encuentra activa y dentro del período establecido para recibir inscripciones.

Regla 6. Validación del estudiante solicitante

Los estudiantes deben encontrarse activos para presentar solicitudes. Asimismo, cada solicitud realizada mediante una cuenta estudiantil debe corresponder al propietario de dicha cuenta.

Regla 7. Validación del motivo de solicitud

El motivo presentado por el estudiante debe cumplir con la longitud mínima establecida. Esta condición permite evitar registros que no contengan una justificación suficiente para iniciar el proceso de evaluación.

Regla 8. Restricción de modificaciones posteriores a la evaluación

Una solicitud que ya fue evaluada no puede modificarse mediante las operaciones disponibles en la API. Esta restricción contribuye a mantener la correspondencia entre la solicitud y la decisión registrada.

Regla 9. Validación del puntaje de evaluación

El puntaje asignado durante una evaluación debe encontrarse entre 0 y 100. Los valores fuera de este intervalo son rechazados por el sistema.

Regla 10. Unicidad de las evaluaciones

Cada solicitud puede tener como máximo una evaluación registrada. Esta restricción evita que se generen múltiples decisiones para una misma solicitud.

Regla 11. Protección de evaluaciones con asignaciones

Una evaluación que ya tiene una asignación de beca relacionada no puede modificarse. Esta condición protege la información utilizada para justificar el otorgamiento del beneficio económico.

Regla 12. Validación del monto mensual

El monto mensual de una asignación debe ser mayor que cero. Los valores negativos o iguales a cero se consideran inválidos.

Regla 13. Condiciones para asignar una beca

Las asignaciones únicamente pueden registrarse cuando la evaluación correspondiente se encuentra aprobada y existen cupos disponibles en la convocatoria relacionada.

Regla 14. Protección de los cupos comprometidos

La cantidad de cupos de una convocatoria no puede reducirse por debajo del número de asignaciones que deben conservarse. Esta validación impide modificaciones que comprometan los beneficios registrados.

Verificación de las reglas de negocio

La comprobación de las reglas de negocio se efectuó mediante solicitudes HTTP ejecutadas desde Postman. Para ello, se utilizaron datos de prueba destinados a provocar respuestas de error controladas y verificar que las operaciones incompatibles con las restricciones fueran rechazadas.

Durante este proceso se realizaron pruebas relacionadas con solicitudes duplicadas, modificaciones de registros evaluados, montos negativos, puntajes fuera del rango permitido, fechas incorrectas, cupos inválidos y evaluaciones duplicadas.

Los resultados obtenidos permitieron comprobar el comportamiento de las validaciones implementadas y la generación de respuestas HTTP coherentes con las condiciones establecidas.

Tabla 2

Resultados de las pruebas manuales de validación de BecaGest

Validación comprobada	Código HTTP	Resultado
Registro de una solicitud duplicada	400	Rechazado
Modificación de una solicitud evaluada	409	Rechazado
Registro de un monto mensual negativo	400	Rechazado
Modificación de una evaluación con asignación	409	Rechazado

Registro de un puntaje superior a 100	400	Rechazado
Convocatoria con fecha final anterior a la inicial	400	Rechazado
Convocatoria con cero cupos	400	Rechazado
Registro de una segunda evaluación para una solicitud	400	Rechazado

Nota. Elaboración propia a partir de los resultados obtenidos mediante Postman durante las pruebas funcionales de BecaGest. El código HTTP 400 Bad Request corresponde a solicitudes con datos inválidos, mientras que 409 Conflict representa operaciones incompatibles con el estado actual de los registros.

Manejo de errores de validación

El manejo de errores en BecaGest se realiza mediante respuestas HTTP que permiten identificar las condiciones que impiden completar una operación.

El código 400 Bad Request se utiliza cuando la información recibida incumple las restricciones de validación. Entre los casos comprobados se encuentran el registro de montos negativos, puntajes superiores al límite permitido, fechas incorrectas y cupos inválidos.

Por otra parte, el código 409 Conflict se utiliza para representar determinadas operaciones que no pueden ejecutarse debido al estado de los registros relacionados. Este comportamiento se comprobó mediante el intento de modificar una solicitud previamente evaluada y una evaluación que ya contaba con una asignación de beca.

Las respuestas generadas incluyen mensajes explicativos que permiten identificar el motivo del rechazo, facilitando la interpretación de los resultados durante las pruebas funcionales.

Importancia de las validaciones

Las reglas de negocio implementadas contribuyen a preservar la coherencia de la información y a mantener un control sobre las operaciones realizadas durante el proceso de gestión de becas.

Estas restricciones permiten evitar duplicidades, proteger decisiones previamente registradas, controlar los beneficios económicos y establecer condiciones para la participación de los estudiantes en las convocatorias.

Asimismo, la aplicación de validaciones durante las operaciones de creación y actualización favorece el mantenimiento de la integridad de los datos almacenados en MySQL.

Los resultados de las pruebas realizadas evidencian el funcionamiento de las ocho condiciones verificadas manualmente y proporcionan información para evaluar la confiabilidad de las operaciones implementadas en BecaGest.

Consultas ORM y recuperación de información

La recuperación y el procesamiento de información constituyen funcionalidades fundamentales para el funcionamiento de BecaGest, debido a que permiten consultar los registros almacenados y obtener datos relacionados con las diferentes etapas de gestión de becas estudiantiles.

Para realizar estas operaciones se utiliza Django Object-Relational Mapper (ORM), una herramienta que permite interactuar con la base de datos mediante instrucciones de Python, utilizando los modelos definidos en el proyecto.

La implementación de Django ORM facilita la consulta de registros, la aplicación de condiciones de filtrado, el acceso a entidades relacionadas y la ejecución de operaciones de agregación. Estas funcionalidades permiten recuperar información relevante sin necesidad de escribir directamente las consultas SQL correspondientes.

En BecaGest se utilizan consultas que relacionan estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones. Asimismo, se emplean operaciones de conteo, sumatoria, filtrado y recuperación optimizada de relaciones para apoyar la generación de información administrativa.

Consultas ORM implementadas

Consulta 1. Cantidad de solicitudes por convocatoria

Esta consulta permite determinar el número de solicitudes registradas para cada convocatoria del sistema.

Su implementación utiliza la función Count junto con annotate, lo que permite agregar un campo calculado que representa la cantidad de solicitudes asociadas con cada convocatoria.

Consulta utilizada:

```
Convocatoria.objects.annotate(
total_solicitudes=Count('solicitudes')
).values('nombre', 'total_solicitudes')
```

La información obtenida permite identificar las convocatorias y conocer la cantidad de solicitudes recibidas en cada una.

Esta consulta resulta útil para el seguimiento de la participación estudiantil y la elaboración de reportes administrativos.

Consulta 2. Solicitudes pendientes de convocatorias activas

La segunda consulta permite recuperar las solicitudes que se encuentran pendientes y pertenecen a convocatorias activas.

Para ello, se aplica el método filter con condiciones relacionadas con el estado de las solicitudes y la disponibilidad de sus convocatorias.

Consulta utilizada:

```
Solicitud.objects.filter(

estado='PENDIENTE',

convocatoria__activa=True

).select_related('estudiante', 'convocatoria')
```

La utilización de select_related permite recuperar información de las entidades relacionadas mediante una consulta optimizada, evitando realizar consultas individuales innecesarias para obtener los datos del estudiante y de la convocatoria.

Esta funcionalidad facilita la identificación de solicitudes pendientes que requieren atención dentro del proceso de gestión de becas.

Consulta 3. Estudiantes con solicitudes aprobadas por carrera

Esta consulta permite identificar las carreras académicas de los estudiantes que cuentan con solicitudes aprobadas y obtener la cantidad de estudiantes correspondiente a cada carrera.

Su implementación combina filtros sobre relaciones, agrupación y funciones de agregación.

Consulta utilizada:

```
Estudiante.objects.filter(

solicitudes__estado='APROBADA'

).values('carrera').annotate(
```

total=Count('id', distinct=True)

)

La función Count permite contabilizar los estudiantes, mientras que distinct=True evita que un mismo estudiante sea contado repetidamente dentro de una carrera cuando dispone de varias solicitudes aprobadas.

Los resultados permiten analizar la distribución de estudiantes beneficiados según su carrera académica.

Consulta 4. Recuperación de estudiantes y sus solicitudes

La cuarta consulta permite recuperar los registros de estudiantes junto con las solicitudes relacionadas.

Su implementación utiliza el método prefetch_related, que facilita la recuperación de relaciones de uno a muchos mediante consultas adicionales organizadas.

Consulta utilizada:

Estudiante.objects.prefetch_related(

'solicitudes'

).order_by('id')

El método prefetch_related contribuye a reducir las consultas repetitivas que podrían producirse al acceder individualmente a las solicitudes de cada estudiante.

Por otra parte, order_by permite establecer un orden de recuperación según el identificador del estudiante.

Esta consulta facilita el análisis conjunto de la información estudiantil y las solicitudes registradas.

Consulta 5. Monto mensual de asignaciones por convocatoria

La quinta consulta permite calcular los montos mensuales asociados con las asignaciones de becas, agrupándolos según la convocatoria correspondiente.

Su implementación utiliza las funciones Sum y Count para obtener el monto acumulado y la cantidad de asignaciones registradas.

Consulta utilizada:

```
Asignacion.objects.exclude(
estado='CANCELADA'
).values(
'evaluacion__solicitud__convocatoria__nombre'
).annotate(
total=Sum('monto_mensual'),
cantidad=Count('id')
)
```

El método exclude permite omitir las asignaciones que presentan el estado CANCELADA.

La función Sum calcula el monto mensual acumulado de las asignaciones consideradas, mientras que Count permite determinar la cantidad de registros asociados con cada convocatoria.

Esta consulta resulta útil para el seguimiento administrativo de los beneficios económicos registrados en BecaGest.

Implementación de filtros REST

Los filtros REST permiten recuperar registros que cumplen condiciones específicas establecidas mediante parámetros enviados en las solicitudes HTTP.

En BecaGest se implementaron filtros combinables en los módulos de estudiantes y convocatorias. Esta funcionalidad permite utilizar diferentes condiciones simultáneamente, facilitando la recuperación de información según las necesidades de consulta.

Filtros del módulo de estudiantes

El módulo de estudiantes permite aplicar condiciones relacionadas con la carrera académica y el estado de actividad del estudiante.

Un ejemplo de consulta es:

```
GET /api/estudiantes/?carrera=Informática&activo=true
```

Esta solicitud permite recuperar los registros correspondientes a estudiantes activos cuya carrera académica sea Informática.

El uso combinado de parámetros permite obtener resultados más específicos que mediante una consulta general.

Filtros del módulo de convocatorias

El módulo de convocatorias permite aplicar filtros relacionados con el estado de actividad y el período de apertura.

Un ejemplo de consulta es:

GET /api/convocatorias/?activa=true&abierta=si

Esta solicitud permite recuperar convocatorias activas cuya fecha actual se encuentre dentro del período establecido para la inscripción de estudiantes.

La combinación de condiciones permite distinguir las convocatorias disponibles de aquellas que se encuentran inactivas o fuera del período de recepción de solicitudes.

Implementación de búsquedas

Las funcionalidades de búsqueda permiten localizar registros utilizando coincidencias parciales sobre determinados campos de texto.

En BecaGest se incorporó esta capacidad mediante el parámetro buscar, utilizado para consultar información relacionada con los recursos disponibles.

En el módulo de convocatorias se puede realizar la siguiente solicitud:

GET /api/convocatorias/?buscar=Transporte

Esta operación permite localizar convocatorias cuyo nombre o descripción contenga el término indicado.

La implementación utiliza búsquedas parciales que facilitan la localización de información sin requerir que el usuario conozca el nombre completo del registro.

Ordenamiento de resultados

El ordenamiento permite establecer la secuencia en la que se presentan los registros recuperados mediante las consultas REST.

En BecaGest se implementó esta funcionalidad en el módulo de convocatorias, utilizando el parámetro orden.

Para obtener las convocatorias ordenadas alfabéticamente de manera ascendente, se utiliza:

GET /api/convocatorias/?orden=nombre

Para obtener los registros en orden descendente, se utiliza:

GET /api/convocatorias/?orden=-nombre

El signo negativo indica que el ordenamiento debe realizarse de manera descendente.

Esta funcionalidad contribuye a mejorar la organización de los resultados y facilita la consulta de información.

Implementación de paginación

La paginación constituye un mecanismo que permite dividir los resultados de una consulta en grupos de registros, evitando presentar toda la información disponible en una sola respuesta.

En BecaGest se implementó la paginación mediante las funcionalidades proporcionadas por Django REST Framework.

Esta característica se utiliza en los listados de estudiantes y convocatorias, con un tamaño predeterminado de cinco registros por página.

Asimismo, el sistema permite indicar el número de página y modificar la cantidad de registros solicitados mediante parámetros HTTP.

Paginación de estudiantes

Un ejemplo de consulta es:

GET /api/estudiantes/?page=1&page_size=1

Esta solicitud permite recuperar la primera página del listado de estudiantes, limitando la respuesta a un registro.

Para consultar la segunda página, se utiliza:

GET /api/estudiantes/?page=2&page_size=1

Paginación de convocatorias

La misma funcionalidad se encuentra disponible en el módulo de convocatorias.

Primera página:

GET /api/convocatorias/?page=1&page_size=1

Segunda página:

GET /api/convocatorias/?page=2&page_size=1

Las respuestas paginadas contienen los campos count, next, previous y results.

El campo count representa la cantidad total de registros encontrados. Los campos next y previous indican las direcciones correspondientes a las páginas siguiente y anterior, cuando existen. Finalmente, results contiene los registros incluidos en la página solicitada.

Verificación de las consultas REST

Las funcionalidades de consulta, filtrado, búsqueda, ordenamiento y paginación se comprobaron mediante solicitudes HTTP ejecutadas desde Postman.

Durante estas verificaciones se utilizaron registros de estudiantes y convocatorias almacenados en MySQL, con el propósito de comprobar que los parámetros enviados fueran procesados correctamente por la API.

Las pruebas incluyeron filtros combinados por carrera y estado de actividad, consultas de convocatorias abiertas, búsquedas por nombre, ordenamiento ascendente y descendente, así como recuperación de registros mediante diferentes páginas.

Las consultas realizadas devolvieron respuestas HTTP 200 OK y permitieron verificar la estructura de los resultados generados por el sistema.

Importancia de las consultas implementadas

La utilización de Django ORM y de las funcionalidades de consulta REST permite administrar de manera organizada la recuperación de información almacenada en BecaGest.

Las operaciones de agregación facilitan la obtención de indicadores administrativos, mientras que los filtros y las búsquedas permiten localizar registros específicos.

Asimismo, las funciones de ordenamiento y paginación contribuyen a presentar la información de forma estructurada y permiten controlar la cantidad de registros incluidos en las respuestas.

En conjunto, estas funcionalidades proporcionan mecanismos de consulta que complementan los procesos de registro, evaluación y asignación de becas estudiantiles.

Pruebas automatizadas y verificación del sistema

Las pruebas de software constituyen una etapa fundamental dentro del proceso de desarrollo de BecaGest, debido a que permiten comprobar el funcionamiento de las operaciones implementadas y determinar si el sistema cumple con las condiciones establecidas para la gestión de becas estudiantiles.

La estrategia de verificación contempló la ejecución de pruebas automatizadas mediante Django y la realización de pruebas manuales utilizando Postman. Ambos mecanismos permitieron evaluar diferentes aspectos del sistema, incluyendo el procesamiento de solicitudes HTTP, las reglas de negocio, las restricciones de acceso y la interacción entre los módulos.

Las pruebas se realizaron sobre una API REST desarrollada con Django REST Framework y configurada para utilizar MySQL como sistema de gestión de bases de datos.

Pruebas automatizadas

Las pruebas automatizadas se implementaron utilizando las funcionalidades de pruebas proporcionadas por Django. Estas permiten ejecutar comprobaciones sobre el comportamiento del sistema y detectar posibles errores durante las operaciones de los módulos.

BecaGest cuenta con un conjunto de 20 pruebas automatizadas, distribuidas entre las cinco aplicaciones principales. Cada módulo incorpora cuatro pruebas, lo que permite mantener una distribución equilibrada de las verificaciones.

La ejecución se realizó desde la terminal del proyecto mediante el siguiente comando:

python manage.py test

Este comando permite identificar y ejecutar las pruebas definidas en las aplicaciones de Django, así como presentar un resumen de los resultados obtenidos.

Durante la ejecución confirmada se completaron satisfactoriamente las 20 pruebas automatizadas.

Distribución de las pruebas automatizadas

Tabla 3

Distribución de las pruebas automatizadas por módulo

Módulo	**Cantidad de pruebas**	**Resultado**
Estudiantes	4	Superadas
Convocatorias	4	Superadas
Solicitudes	4	Superadas
Evaluaciones	4	Superadas
Asignaciones	4	Superadas
Total	**20**	**Superadas**

Nota. Elaboración propia a partir de la ejecución de las pruebas automatizadas del proyecto BecaGest mediante el comando python manage.py test.

La distribución de las pruebas permite comprobar el funcionamiento de los diferentes componentes de la aplicación y proporciona un mecanismo para identificar posibles errores cuando se realizan modificaciones en el código.

El resultado satisfactorio de la ejecución representa una evidencia del comportamiento esperado en los casos incluidos dentro del conjunto de pruebas. Sin embargo, este resultado no implica que se hayan evaluado todas las situaciones posibles del sistema.

Pruebas manuales mediante Postman

Además de las pruebas automatizadas, se realizaron comprobaciones manuales utilizando Postman, una herramienta que permite enviar solicitudes HTTP y analizar las respuestas generadas por una API REST.

Estas pruebas permitieron verificar las funcionalidades del sistema desde la perspectiva de un cliente que consume los servicios disponibles.

Para su ejecución se utilizaron diferentes métodos HTTP, entre ellos GET, POST, PUT, PATCH y DELETE, de acuerdo con las operaciones admitidas por cada recurso.

Asimismo, se utilizaron tokens de autenticación correspondientes a los diferentes perfiles del sistema, lo que permitió comprobar los controles de acceso y las restricciones aplicadas a determinadas funcionalidades.

Las pruebas manuales comprendieron operaciones de consulta, creación, modificación y eliminación de registros, así como validaciones de datos, consultas filtradas y comprobaciones de autorización.

Verificación de operaciones CRUD

Las operaciones CRUD corresponden a las funcionalidades necesarias para crear, consultar, actualizar y eliminar registros dentro de un sistema.

En BecaGest, estas operaciones se implementaron en los cinco módulos principales y se comprobaron mediante solicitudes realizadas desde Postman.

Las operaciones de consulta permitieron recuperar información almacenada en MySQL y verificar la estructura de las respuestas JSON.

Las operaciones de creación se utilizaron para registrar nuevos recursos, considerando las condiciones establecidas por las reglas de negocio y los permisos correspondientes.

Las operaciones de actualización permitieron comprobar el comportamiento de los métodos PUT y PATCH, incluyendo las restricciones aplicadas a registros que ya habían avanzado dentro del proceso de gestión de becas.

Finalmente, las operaciones de eliminación permitieron verificar la respuesta del sistema cuando un recurso podía eliminarse y cuando existían condiciones que impedían ejecutar dicha acción.

Verificación de la integración entre módulos

La integración funcional de BecaGest se comprobó mediante el registro de información relacionada con las distintas etapas del proceso de gestión de becas estudiantiles.

Inicialmente, se utilizaron estudiantes y convocatorias registrados en el sistema para generar las solicitudes correspondientes.

Posteriormente, se verificó el proceso de evaluación de una solicitud y la actualización de su estado según el resultado registrado.

En uno de los casos comprobados, una solicitud aprobada quedó relacionada con una evaluación y una asignación de beneficio económico.

La asignación registrada permitió verificar la relación entre los módulos de solicitudes, evaluaciones y asignaciones.

Esta comprobación evidenció que los registros pueden relacionarse correctamente mediante las claves foráneas establecidas en el modelo de datos.

Asimismo, permitió observar el funcionamiento del proceso que conecta la presentación de una solicitud con la aprobación y el registro de un beneficio económico.

Verificación de filtros, búsquedas y paginación

Las pruebas manuales también contemplaron la ejecución de consultas REST que utilizan parámetros para recuperar información específica.

Se verificó el funcionamiento de filtros combinados en el módulo de estudiantes, utilizando condiciones relacionadas con la carrera académica y el estado de actividad.

En el módulo de convocatorias se comprobaron filtros que permiten recuperar convocatorias activas y abiertas, además de búsquedas por nombre y ordenamiento de resultados.

La paginación se evaluó mediante los parámetros page y page_size, comprobando la recuperación de registros distribuidos en diferentes páginas.

Estas pruebas permitieron verificar que el sistema procesa los parámetros enviados y organiza los resultados de acuerdo con las condiciones solicitadas.

Verificación de las reglas de negocio

Las pruebas de validación se orientaron a comprobar que el sistema rechazara operaciones incompatibles con las condiciones establecidas para la gestión de becas.

Entre los casos evaluados se incluyeron el registro de solicitudes duplicadas, modificaciones de solicitudes evaluadas, montos mensuales negativos y puntajes fuera del rango permitido.

También se comprobaron las restricciones relacionadas con las fechas de las convocatorias, la cantidad de cupos disponibles y la existencia de evaluaciones previamente registradas.

Los resultados permitieron verificar que las operaciones inválidas fueran rechazadas mediante respuestas HTTP y mensajes de error.

Este comportamiento contribuye a prevenir inconsistencias en los registros y a mantener las condiciones establecidas para el procesamiento de solicitudes y asignaciones.

Verificación de autenticación y permisos

La seguridad del sistema se evaluó mediante solicitudes realizadas con diferentes condiciones de autenticación y autorización.

Una de las comprobaciones consistió en consultar el recurso de perfil sin proporcionar un token de autenticación. Como resultado, el sistema devolvió el código HTTP 401 Unauthorized.

Asimismo, se verificó que un usuario con el perfil Estudiante no pudiera registrar evaluaciones ni consultar reportes administrativos. Estas operaciones generaron respuestas HTTP 403 Forbidden.

Posteriormente, se realizó una consulta al reporte administrativo utilizando una cuenta con el perfil Encargado. La operación fue autorizada y devolvió una respuesta HTTP 200 OK.

Estos resultados permitieron comprobar que el sistema diferencia entre solicitudes sin autenticación, usuarios autenticados sin permisos suficientes y usuarios autorizados para acceder a determinados recursos.

Verificación de códigos de respuesta HTTP

Las respuestas HTTP permiten comunicar el resultado de las operaciones realizadas mediante la API REST.

Durante las pruebas manuales de BecaGest se comprobaron ocho códigos de estado, correspondientes a operaciones satisfactorias y situaciones de error.

Tabla 4

Códigos de respuesta HTTP verificados en BecaGest

Código HTTP	Descripción	Ejemplo de comprobación
200 OK	Operación satisfactoria	Consulta del reporte administrativo
201 Created	Recurso creado	Registro de una convocatoria temporal
204 No Content	Eliminación satisfactoria	Eliminación de una convocatoria temporal
400 Bad Request	Datos inválidos	Registro de cupos iguales a cero

401 Unauthorized	Autenticación requerida	Consulta de perfil sin token
403 Forbidden	Acceso no autorizado	Estudiante intenta registrar una evaluación
404 Not Found	Recurso inexistente	Consulta de convocatoria con identificador 9999
409 Conflict	Conflicto con el estado del recurso	Modificación de una solicitud evaluada

Nota. Elaboración propia a partir de las respuestas obtenidas durante las pruebas manuales realizadas mediante Postman.

La comprobación de estos códigos permitió verificar que las respuestas del sistema fueran coherentes con las operaciones ejecutadas y con las restricciones de acceso y validación implementadas.

Resultados de las pruebas

Los resultados obtenidos durante el proceso de verificación permitieron comprobar el funcionamiento de las principales funcionalidades de BecaGest.

La ejecución de las 20 pruebas automatizadas finalizó satisfactoriamente, mientras que las comprobaciones manuales permitieron evaluar el comportamiento de los endpoints y las respuestas HTTP generadas por el sistema.

Las pruebas de integración evidenciaron la relación funcional entre los módulos de estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones.

Por otra parte, las verificaciones de seguridad confirmaron la aplicación de restricciones de acceso según los perfiles de usuario, mientras que las pruebas de validación permitieron observar el rechazo de operaciones que incumplían las reglas de negocio.

Los resultados obtenidos constituyen evidencia del cumplimiento de las funcionalidades evaluadas dentro del alcance académico del proyecto.

Sin embargo, estas comprobaciones no representan una evaluación exhaustiva de todas las condiciones posibles de funcionamiento, seguridad o rendimiento en un entorno de producción.

Conclusión de la verificación

El proceso de pruebas permitió evaluar las principales operaciones de BecaGest mediante mecanismos automatizados y manuales.

La combinación de ambos procedimientos proporcionó evidencia sobre el funcionamiento de los módulos, la aplicación de reglas de negocio, las restricciones de seguridad y las respuestas HTTP.

La ejecución satisfactoria de las 20 pruebas automatizadas, junto con las verificaciones realizadas en Postman, respalda el cumplimiento de las funcionalidades evaluadas dentro del alcance académico del proyecto.

En conjunto, los resultados obtenidos permiten establecer que BecaGest dispone de mecanismos de verificación que contribuyen a identificar errores y mantener un funcionamiento consistente durante las operaciones de gestión de becas estudiantiles.

Catálogo de endpoints y documentación de la API REST

La implementación de BecaGest se fundamenta en una arquitectura de servicios REST que permite la comunicación entre los usuarios y las funcionalidades del sistema mediante solicitudes HTTP. Esta arquitectura facilita el acceso controlado a los recursos correspondientes a estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones.

Los servicios fueron desarrollados utilizando Django REST Framework, que proporciona herramientas para procesar solicitudes, validar información y generar respuestas estructuradas en formato JSON.

Cada endpoint representa una ruta de acceso a una funcionalidad específica del sistema. Las operaciones disponibles se encuentran asociadas con métodos HTTP y sujetas a los mecanismos de autenticación, autorización y validación establecidos.

Estructura general de la API

La API de BecaGest se encuentra organizada mediante rutas que utilizan el prefijo /api/, seguido del nombre del recurso correspondiente.

Durante el desarrollo y las pruebas locales se utilizó la siguiente dirección base:

http://127.0.0.1:8000

A partir de esta dirección se accede a los diferentes endpoints implementados.

La estructura permite identificar claramente los recursos y las operaciones relacionadas con cada módulo.

Las solicitudes realizadas mediante Postman utilizan métodos HTTP específicos y, cuando corresponde, incluyen datos en formato JSON y un token de autenticación.

Catálogo de endpoints

Tabla 5

Catálogo de endpoints implementados en la API REST de BecaGest

Método HTTP	**Endpoint**	**Funcionalidad**
POST	/api/token/	Obtener un token de autenticación.
GET	/api/perfil/	Consultar los datos del usuario autenticado.
GET	/api/sesion/	Consultar información de la sesión.
GET	/api/reportes/resumen/	Obtener estadísticas administrativas.
GET	/api/estudiantes/	Consultar el listado de estudiantes.
POST	/api/estudiantes/	Registrar un estudiante.
GET	/api/estudiantes/{id}/	Consultar un estudiante específico.
PUT, PATCH	/api/estudiantes/{id}/	Actualizar información estudiantil.
DELETE	/api/estudiantes/{id}/	Eliminar un estudiante.
GET	/api/convocatorias/	Consultar las convocatorias.
POST	/api/convocatorias/	Registrar una convocatoria.
GET	/api/convocatorias/{id}/	Consultar una convocatoria específica.
PUT, PATCH	/api/convocatorias/{id}/	Actualizar una convocatoria.

DELETE	/api/convocatorias/{id}/	Eliminar una convocatoria.
GET	/api/solicitudes/	Consultar las solicitudes registradas.
POST	/api/solicitudes/	Registrar una solicitud.
GET	/api/solicitudes/{id}/	Consultar una solicitud específica.
PUT, PATCH	/api/solicitudes/{id}/	Actualizar una solicitud pendiente.
DELETE	/api/solicitudes/{id}/	Eliminar una solicitud permitida.
GET	/api/evaluaciones/	Consultar las evaluaciones.
POST	/api/evaluaciones/	Registrar una evaluación.
GET	/api/evaluaciones/{id}/	Consultar una evaluación específica.
PUT, PATCH	/api/evaluaciones/{id}/	Actualizar una evaluación permitida.
DELETE	/api/evaluaciones/{id}/	Eliminar una evaluación permitida.
GET	/api/asignaciones/	Consultar las asignaciones.
POST	/api/asignaciones/	Registrar una asignación económica.
GET	/api/asignaciones/{id}/	Consultar una asignación específica.
PUT, PATCH	/api/asignaciones/{id}/	Actualizar una asignación.
DELETE	/api/asignaciones/{id}/	Eliminar una asignación permitida.

Nota. Elaboración propia a partir de las rutas implementadas en BecaGest. El parámetro {id} representa el identificador del registro. La disponibilidad de las operaciones depende del perfil autenticado y del cumplimiento de las reglas de negocio.

Métodos HTTP implementados

La API REST utiliza diferentes métodos HTTP para ejecutar operaciones sobre los recursos disponibles.

El método GET permite recuperar información almacenada en la base de datos. Este método se utiliza para consultar registros individuales, listados y reportes administrativos.

El método POST permite crear registros y enviar información necesaria para ejecutar operaciones específicas, como la obtención de tokens de autenticación.

El método PUT se utiliza para actualizar un recurso mediante el envío de los datos correspondientes a su representación.

El método PATCH permite modificar parcialmente un registro, actualizando únicamente los campos indicados en la solicitud.

Finalmente, el método DELETE permite eliminar registros cuando se cumplen las condiciones de autorización e integridad establecidas.

La utilización de estos métodos contribuye a mantener una estructura uniforme para las operaciones de la API.

Documentación del módulo de estudiantes

Los endpoints del módulo de estudiantes permiten registrar, consultar y administrar la información correspondiente a los usuarios estudiantiles.

El endpoint /api/estudiantes/ permite consultar el listado general de estudiantes para los perfiles autorizados y registrar nuevos estudiantes cuando la operación es ejecutada por un administrador.

Por otra parte, /api/estudiantes/{id}/ permite recuperar la información correspondiente a un registro específico y realizar modificaciones o eliminaciones cuando se dispone de autorización.

Las operaciones de actualización y eliminación se encuentran restringidas al perfil Administrador.

Asimismo, el sistema impide eliminar estudiantes que poseen solicitudes relacionadas, preservando la integridad referencial de los registros.

Documentación del módulo de convocatorias

Los endpoints de convocatorias permiten administrar la información correspondiente a los períodos de inscripción y disponibilidad de beneficios económicos.

La ruta /api/convocatorias/ proporciona acceso al listado de convocatorias registradas y permite crear nuevos registros cuando la operación es ejecutada por un administrador.

La ruta /api/convocatorias/{id}/ permite consultar, modificar o eliminar una convocatoria específica.

Las operaciones de modificación se encuentran sujetas a validaciones relacionadas con las fechas y la cantidad de cupos disponibles.

Asimismo, se aplican restricciones para evitar la eliminación de convocatorias que poseen solicitudes asociadas.

Documentación del módulo de solicitudes

Los endpoints del módulo de solicitudes permiten gestionar las peticiones de becas estudiantiles.

La ruta /api/solicitudes/ permite consultar solicitudes y registrar nuevas peticiones, considerando los permisos correspondientes al usuario autenticado.

Las cuentas estudiantiles únicamente pueden registrar solicitudes relacionadas con su propia información.

La ruta /api/solicitudes/{id}/ permite consultar los datos de una solicitud y ejecutar operaciones de actualización o eliminación cuando las condiciones del sistema lo permiten.

Una vez evaluada una solicitud, las restricciones implementadas impiden que sea modificada o eliminada mediante las operaciones disponibles en la API.

Además, el registro de solicitudes se encuentra condicionado por la disponibilidad de la convocatoria, el estado del estudiante y la restricción que impide duplicar solicitudes para una misma convocatoria.

Documentación del módulo de evaluaciones

El módulo de evaluaciones permite registrar y consultar los resultados de las solicitudes estudiantiles.

La ruta /api/evaluaciones/ proporciona acceso a las evaluaciones registradas y permite crear nuevos resultados mediante cuentas autorizadas.

La creación de evaluaciones está restringida a los perfiles Administrador y Encargado.

La ruta /api/evaluaciones/{id}/ permite consultar evaluaciones específicas y realizar modificaciones cuando las condiciones establecidas lo permiten.

El registro de una evaluación actualiza el estado de la solicitud relacionada, manteniendo la correspondencia entre ambos recursos.

Asimismo, el sistema impide modificar evaluaciones que ya poseen una asignación económica asociada.

Documentación del módulo de asignaciones

Los endpoints del módulo de asignaciones permiten administrar los beneficios económicos otorgados a los estudiantes.

La ruta /api/asignaciones/ permite consultar las asignaciones existentes y registrar nuevos beneficios económicos mediante usuarios autorizados.

La ruta /api/asignaciones/{id}/ permite consultar, actualizar o eliminar asignaciones, considerando las restricciones correspondientes.

Las operaciones de registro se encuentran condicionadas por la existencia de una evaluación aprobada, la disponibilidad de cupos y la validez del monto mensual.

Las asignaciones registradas pueden presentar los estados ACTIVA, FINALIZADA y CANCELADA, los cuales permiten identificar la situación del beneficio económico.

Endpoints de autenticación y administración

BecaGest incorpora endpoints complementarios destinados a proporcionar funcionalidades de autenticación, consulta de perfil, manejo de sesiones y generación de información administrativa.

El endpoint /api/token/ permite obtener el token utilizado para autenticar solicitudes posteriores.

El endpoint /api/perfil/ permite recuperar información correspondiente al usuario autenticado, incluyendo sus datos de identificación y grupos asociados.

El endpoint /api/sesion/ proporciona una funcionalidad de manejo de sesiones, mediante la cual se registra información relacionada con las visitas efectuadas durante una misma sesión.

Finalmente, /api/reportes/resumen/ permite consultar indicadores administrativos relacionados con estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones.

El acceso a los reportes se encuentra restringido a los perfiles Administrador y Encargado.

Parámetros de consulta disponibles

La API permite utilizar parámetros adicionales para filtrar, buscar, ordenar y paginar determinados conjuntos de registros.

Tabla 6

Parámetros de consulta implementados en BecaGest

Parámetro	**Aplicación**	**Funcionalidad**
carrera	Estudiantes	Filtrar por carrera académica.
activo	Estudiantes	Filtrar por estado de actividad.
buscar	Estudiantes y convocatorias	Localizar coincidencias de texto.
activa	Convocatorias	Filtrar convocatorias activas o inactivas.
abierta	Convocatorias	Consultar convocatorias abiertas.
orden	Convocatorias	Ordenar los resultados.
estado	Solicitudes y asignaciones	Filtrar según el estado.
convocatoria	Solicitudes	Filtrar por convocatoria relacionada.
estudiante	Solicitudes	Filtrar por estudiante relacionado.
page	Estudiantes y convocatorias	Seleccionar el número de página.
page_size	Estudiantes y convocatorias	Establecer la cantidad de resultados por página.

Nota. Elaboración propia a partir de los parámetros de consulta implementados en los endpoints de BecaGest.

Los parámetros permiten recuperar conjuntos de información más específicos y contribuyen a mejorar la organización de los resultados obtenidos mediante solicitudes HTTP.

Estructura de las solicitudes y respuestas

Las operaciones de creación y actualización utilizan información estructurada en formato JSON. Este formato permite representar los atributos de los recursos mediante pares de claves y valores.

La información recibida se procesa mediante los serializadores de Django REST Framework, que permiten validar los campos y convertir los datos a las representaciones correspondientes.

Una vez ejecutada la operación, la API devuelve una respuesta HTTP acompañada de los datos solicitados o de mensajes relacionados con el resultado del procesamiento.

Las respuestas satisfactorias pueden incluir registros individuales, listados, información estadística o confirmaciones de operaciones ejecutadas.

Por otra parte, las solicitudes que incumplen las restricciones establecidas generan respuestas de error que permiten identificar las condiciones que impidieron completar la operación.

Seguridad de los endpoints

La protección de los endpoints se encuentra relacionada con los mecanismos de autenticación y autorización implementados en Django REST Framework.

Las operaciones protegidas requieren credenciales válidas mediante tokens de autenticación. Posteriormente, se verifica que el usuario disponga de los permisos necesarios para ejecutar la acción solicitada.

Los perfiles Administrador, Encargado y Estudiante presentan diferentes niveles de acceso, de acuerdo con las funciones asignadas dentro del sistema.

Las consultas de información privada también consideran la relación entre el usuario autenticado y los registros solicitados, evitando que las cuentas estudiantiles accedan a información que no les corresponde.

Estos controles contribuyen a mantener la confidencialidad de los datos y a restringir las modificaciones a usuarios autorizados.

Importancia de la documentación de la API

La documentación de los endpoints permite identificar las funcionalidades disponibles y comprender la manera en que los componentes del sistema se comunican mediante solicitudes HTTP.

El catálogo de rutas proporciona una referencia de los recursos implementados, sus métodos de acceso y las operaciones correspondientes.

Asimismo, la descripción de los parámetros y las restricciones permite comprender las condiciones bajo las cuales se procesan las solicitudes.

En conjunto, la documentación de la API REST contribuye a facilitar la comprensión de BecaGest, su mantenimiento y la verificación de las funcionalidades desarrolladas dentro del alcance académico del proyecto.

Instalación, configuración y ejecución del sistema

La instalación y configuración del entorno de desarrollo constituye una etapa fundamental para garantizar el funcionamiento de BecaGest. Durante este proceso se establecieron las herramientas, dependencias y configuraciones necesarias para ejecutar la API REST y establecer la comunicación con la base de datos MySQL.

El sistema fue desarrollado utilizando Python como lenguaje de programación, Django como framework principal y Django REST Framework para la implementación de los servicios REST. Asimismo, se utilizó MySQL como sistema de gestión de bases de datos y Visual Studio Code como entorno de desarrollo.

La configuración del proyecto se realizó en un entorno local con Windows 11, utilizando PowerShell para ejecutar los comandos de instalación, administración y verificación.

Requisitos tecnológicos

BecaGest requiere diferentes herramientas de software para su instalación y funcionamiento. Estas permiten ejecutar el servidor, administrar las dependencias, mantener la información y comprobar las operaciones disponibles.

Tabla 7

Herramientas y tecnologías utilizadas en BecaGest

Tecnología	Función
Python	Lenguaje de programación del sistema.
Django	Framework para el desarrollo del backend.

Django REST Framework	Implementación de la API REST.
MySQL	Almacenamiento y administración de datos.
mysqlclient	Conexión entre Django y MySQL.
python-dotenv	Lectura de variables de entorno.
Visual Studio Code	Entorno de desarrollo y edición del código.
PowerShell	Ejecución de comandos del proyecto.
Postman	Verificación de solicitudes y respuestas HTTP.

Nota. Elaboración propia a partir de las herramientas empleadas durante el desarrollo y las pruebas de BecaGest.

La combinación de estas tecnologías permite establecer un entorno de trabajo orientado al desarrollo de servicios backend y al almacenamiento de información mediante una base de datos relacional.

Configuración del entorno virtual

El proyecto utiliza un entorno virtual de Python para mantener las dependencias necesarias separadas de otras instalaciones existentes en el equipo.

Este mecanismo contribuye a organizar las bibliotecas utilizadas y permite mantener una configuración específica para el desarrollo de BecaGest.

La creación del entorno virtual se realiza mediante el siguiente comando:

```
py -m venv .venv
```

Posteriormente, el entorno puede activarse desde PowerShell mediante:

```
.venv\Scripts\Activate.ps1
```

Una vez activado, las bibliotecas necesarias pueden instalarse a partir del archivo requirements.txt.

El comando utilizado es:

```
python -m pip install -r requirements.txt
```

El archivo requirements.txt contiene las dependencias del proyecto y sus versiones, lo que facilita la reproducción del entorno de ejecución en otros equipos.

Configuración de la base de datos MySQL

BecaGest utiliza MySQL como sistema de gestión de bases de datos relacional. Esta tecnología permite almacenar los registros correspondientes a estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones, además de las tablas requeridas por Django para administrar usuarios, grupos y permisos.

La base de datos utilizada en el proyecto se denomina becagest_db.

La conexión se establece mediante la configuración definida en Django, utilizando el controlador mysqlclient para permitir la comunicación entre el framework y MySQL.

Los parámetros de conexión incluyen el nombre de la base de datos, el usuario autorizado, la contraseña, el servidor y el puerto.

La configuración permite ejecutar operaciones de registro, consulta, modificación y eliminación de información mediante los modelos de Django y su sistema de mapeo objeto-relacional.

Administración de variables de entorno

La configuración de BecaGest incorpora variables de entorno para administrar determinados parámetros de conexión sin incluir directamente las credenciales en el código fuente.

El proyecto dispone de un archivo .env.example, que contiene la estructura de las variables requeridas para configurar el entorno local.

Entre las variables utilizadas se encuentran:

- DB_NAME: nombre de la base de datos.
- DB_USER: usuario autorizado para establecer la conexión.
- DB_PASSWORD: contraseña correspondiente al usuario de MySQL.
- DB_HOST: dirección del servidor de base de datos.
- DB_PORT: puerto utilizado para la conexión.

Los valores específicos se establecen en un archivo .env local, cuya información es recuperada mediante las funcionalidades de python-dotenv.

Esta separación contribuye a evitar la exposición de credenciales y facilita la configuración del sistema en diferentes equipos.

Creación de tablas y migraciones

Django utiliza un sistema de migraciones para administrar los cambios realizados en los modelos y mantener su correspondencia con la estructura de la base de datos.

Las migraciones permiten crear las tablas necesarias, establecer relaciones y aplicar modificaciones al esquema de almacenamiento.

Entre los comandos utilizados para administrar este proceso se encuentran:

python manage.py makemigrations

python manage.py migrate

El primer comando permite generar archivos de migración a partir de los cambios detectados en los modelos. El segundo aplica las migraciones pendientes a la base de datos configurada.

Mediante este procedimiento se establece la estructura relacional necesaria para almacenar la información de los cinco módulos funcionales de BecaGest.

Asimismo, se crean las tablas correspondientes a los componentes de autenticación, permisos y administración proporcionados por Django y las aplicaciones instaladas.

Configuración de usuarios y permisos

La administración de usuarios se realiza mediante las funcionalidades de autenticación proporcionadas por Django.

Para establecer una cuenta administrativa inicial, se utiliza el siguiente comando:

python manage.py createsuperuser

Este procedimiento permite registrar un superusuario con acceso a las funcionalidades administrativas del framework.

Adicionalmente, BecaGest utiliza los grupos Administrador, Encargado y Estudiante para diferenciar las responsabilidades y operaciones autorizadas.

Los grupos se relacionan con los usuarios registrados y permiten organizar los perfiles de acceso definidos para el funcionamiento del sistema.

Las restricciones de autorización se complementan con verificaciones implementadas en las vistas de la API REST, considerando tanto el perfil autenticado como la relación del usuario con los recursos solicitados.

Verificación de la configuración

La configuración general del proyecto puede comprobarse mediante las herramientas de administración proporcionadas por Django.

Uno de los comandos utilizados es:

python manage.py check

Esta instrucción permite ejecutar las comprobaciones del framework para identificar determinados problemas de configuración y estructura del proyecto.

También se utiliza el sistema de migraciones para verificar la correspondencia entre los modelos implementados y la base de datos.

Estas comprobaciones forman parte del proceso de preparación del entorno y contribuyen a identificar errores que podrían afectar la ejecución de la aplicación.

Ejecución del servidor de desarrollo

La ejecución de BecaGest se realiza mediante el servidor de desarrollo integrado en Django.

El comando utilizado es:

python manage.py runserver

De manera predeterminada, el servidor permite acceder a la aplicación desde la siguiente dirección local:

http://127.0.0.1:8000/

A partir de esta dirección se encuentran disponibles las rutas configuradas para los diferentes recursos de la API REST.

El servidor procesa las solicitudes HTTP, verifica las condiciones de autenticación y autorización, ejecuta las operaciones correspondientes y genera las respuestas necesarias.

Este mecanismo permite comprobar el funcionamiento de los módulos durante el desarrollo y las pruebas locales.

El servidor integrado de Django está destinado a actividades de desarrollo y no constituye una configuración de producción.

Ejecución y comprobación de los servicios REST

La interacción con los servicios de BecaGest se realizó mediante Postman, utilizando la dirección local del servidor y las rutas correspondientes a los recursos implementados.

Las operaciones protegidas requieren un token de autenticación válido, obtenido mediante el endpoint /api/token/.

Posteriormente, las solicitudes autorizadas pueden consultar, registrar, modificar o eliminar información, de acuerdo con los permisos y las reglas de negocio correspondientes.

Las respuestas se generan en formato JSON, lo que permite interpretar los datos procesados y verificar los resultados de cada operación.

El uso de Postman facilitó la comprobación de las funcionalidades implementadas, incluyendo las validaciones, los filtros, la paginación, los reportes administrativos y las restricciones de acceso.

Ejecución de las pruebas automatizadas

El sistema incorpora pruebas automatizadas destinadas a verificar las operaciones implementadas en los diferentes módulos.

Estas pruebas pueden ejecutarse mediante el comando:

python manage.py test

Django identifica las pruebas definidas, prepara el entorno de comprobación y ejecuta los casos correspondientes.

En la ejecución verificada del proyecto se completaron satisfactoriamente 20 pruebas automatizadas, distribuidas entre estudiantes, convocatorias, solicitudes, evaluaciones y asignaciones.

Los resultados obtenidos proporcionan evidencia del funcionamiento de los casos evaluados y contribuyen al proceso de verificación del sistema.

Organización de los archivos del proyecto

La estructura de BecaGest se encuentra organizada mediante aplicaciones independientes que representan los principales componentes funcionales del sistema.

Cada aplicación contiene los archivos necesarios para definir modelos, serializadores, vistas, rutas y pruebas, de acuerdo con las responsabilidades correspondientes.

El proyecto también incluye archivos generales de configuración que permiten administrar las aplicaciones instaladas, las conexiones de base de datos y las rutas principales.

Entre los archivos complementarios se encuentran requirements.txt, utilizado para administrar las dependencias; .env.example, que contiene la estructura de configuración; y README.md, destinado a documentar las características y procedimientos generales del sistema.

Esta organización contribuye a mantener una estructura comprensible y facilita la identificación de los componentes utilizados durante el desarrollo.

Consideraciones de seguridad y mantenimiento

La configuración del sistema contempla medidas destinadas a proteger las credenciales y restringir el acceso a la información.

La utilización de variables de entorno permite separar determinados datos sensibles del código fuente, mientras que los mecanismos de autenticación y autorización controlan las operaciones disponibles para cada usuario.

Asimismo, la organización modular permite realizar modificaciones sobre componentes específicos sin alterar directamente las responsabilidades de las demás aplicaciones.

Las migraciones contribuyen al mantenimiento de la estructura de datos, mientras que las pruebas automatizadas proporcionan mecanismos para comprobar el comportamiento de las funcionalidades cuando se realizan cambios en el proyecto.

Conclusión del proceso de instalación y ejecución

El proceso de instalación, configuración y ejecución de BecaGest permitió establecer un entorno de desarrollo funcional para la implementación y verificación de la API REST.

La integración de Python, Django, Django REST Framework y MySQL proporcionó los componentes necesarios para desarrollar las operaciones relacionadas con la gestión de becas estudiantiles.

La utilización de entornos virtuales, variables de configuración, migraciones y mecanismos de autenticación contribuyó a organizar el funcionamiento del sistema y a mantener la separación entre sus diferentes componentes.

Finalmente, la ejecución del servidor local y las verificaciones realizadas mediante Postman y Django permitieron comprobar las funcionalidades desarrolladas dentro del alcance académico del proyecto.

Conclusiones y recomendaciones

Conclusiones

El desarrollo de BecaGest permitió implementar una API REST orientada a la administración de becas estudiantiles, integrando funcionalidades para el registro de estudiantes, la gestión de convocatorias, la presentación de solicitudes, la evaluación de expedientes y la asignación de beneficios económicos. La organización del sistema en cinco módulos funcionales permitió establecer una estructura que facilita la separación de responsabilidades y la comunicación entre los componentes.

La utilización de Django y Django REST Framework proporcionó las herramientas necesarias para implementar operaciones de creación, consulta, actualización y eliminación de registros mediante solicitudes HTTP. Asimismo, el uso de MySQL permitió almacenar la información mediante un modelo relacional que incorpora claves primarias, claves foráneas, restricciones de unicidad y mecanismos de integridad referencial.

La implementación de relaciones entre las entidades permitió mantener la correspondencia entre los diferentes procesos de gestión de becas. De esta manera, las solicitudes se encuentran asociadas con estudiantes y convocatorias, mientras que las evaluaciones y asignaciones permiten registrar las decisiones y los beneficios económicos correspondientes.

En relación con la seguridad, la incorporación de autenticación mediante tokens y autorización basada en los perfiles Administrador, Encargado y Estudiante permitió establecer diferentes niveles de acceso. Estos mecanismos contribuyeron a restringir las operaciones administrativas y a proteger la información asociada con los usuarios.

Las reglas de negocio implementadas permitieron controlar situaciones como el registro de solicitudes duplicadas, la modificación de expedientes evaluados, el ingreso de puntajes fuera del intervalo permitido y la asignación de beneficios sin cumplir las condiciones establecidas.

Estas restricciones favorecen la consistencia de los datos y permiten mantener un control sobre las operaciones realizadas.

La implementación de consultas mediante Django ORM facilitó la recuperación de información relacionada con los diferentes módulos. Las operaciones de agregación, filtrado y consulta de relaciones permitieron obtener datos útiles para el seguimiento administrativo, mientras que las funcionalidades de búsqueda, ordenamiento y paginación contribuyeron a organizar los resultados de la API.

El proceso de verificación permitió comprobar el funcionamiento de las características desarrolladas mediante pruebas automatizadas y manuales. La ejecución de 20 pruebas automatizadas finalizó satisfactoriamente, mientras que las comprobaciones realizadas mediante Postman permitieron evaluar operaciones CRUD, reglas de negocio, restricciones de seguridad y respuestas HTTP.

En términos generales, los resultados obtenidos evidencian que BecaGest cumple con las principales funcionalidades evaluadas dentro del alcance académico establecido. El proyecto permitió integrar conocimientos relacionados con desarrollo backend, bases de datos relacionales, programación orientada a objetos, servicios REST, autenticación y pruebas de software.

Recomendaciones

El desarrollo de BecaGest establece una base funcional que puede ampliarse mediante la incorporación de nuevas características orientadas a mejorar la experiencia de los usuarios y fortalecer los procesos administrativos.

Una de las principales recomendaciones consiste en desarrollar una interfaz gráfica que permita a los estudiantes y al personal administrativo interactuar con las funcionalidades del sistema sin depender directamente de herramientas como Postman. Esta mejora facilitaría la consulta de convocatorias, el registro de solicitudes y el seguimiento de los beneficios económicos.

También se considera conveniente ampliar las funcionalidades de notificación para informar a los estudiantes sobre la apertura de convocatorias, los cambios de estado de sus solicitudes y los resultados de las evaluaciones. La incorporación de este mecanismo permitiría mejorar la comunicación entre los usuarios y el sistema.

En relación con la seguridad, se recomienda fortalecer los mecanismos de protección mediante el uso de conexiones HTTPS, una configuración específica para entornos de producción y políticas de administración de credenciales. Asimismo, podrían incorporarse mecanismos adicionales para el seguimiento de operaciones administrativas y el registro de actividades relevantes.

Otra posibilidad de mejora consiste en ampliar los reportes administrativos mediante indicadores estadísticos y representaciones gráficas. Estas funcionalidades permitirían analizar la distribución de solicitudes, la cantidad de estudiantes beneficiados y los montos económicos asociados con las diferentes convocatorias.

En el ámbito de las pruebas, se recomienda ampliar progresivamente la cobertura automatizada para incorporar escenarios adicionales relacionados con errores de autenticación, límites de cupos, modificaciones simultáneas y comportamiento ante situaciones excepcionales. Esta ampliación contribuiría a fortalecer la confiabilidad del sistema durante futuras modificaciones.

Finalmente, se considera conveniente mantener actualizada la documentación técnica, las dependencias y los procedimientos de ejecución, con el propósito de facilitar el mantenimiento del proyecto y su posible evolución hacia una solución de mayor alcance.

Limitaciones del sistema

BecaGest fue desarrollado como un proyecto académico orientado a demostrar la implementación de una API REST para la gestión de becas estudiantiles. Por esta razón, su alcance se concentra en las funcionalidades establecidas para los cinco módulos principales y en los mecanismos necesarios para comprobar su funcionamiento.

Una de sus limitaciones es la ausencia de una interfaz gráfica independiente para los estudiantes y administradores. La interacción con los servicios se realiza principalmente mediante solicitudes HTTP, utilizando Postman durante las actividades de desarrollo y verificación.

Asimismo, el sistema se ejecuta en un entorno local de desarrollo mediante el servidor integrado de Django, por lo que no cuenta con una infraestructura de despliegue destinada a usuarios externos.

Las pruebas realizadas permitieron verificar las funcionalidades seleccionadas dentro del alcance del proyecto, pero no constituyen una evaluación exhaustiva del rendimiento, la seguridad o la capacidad de respuesta ante una cantidad elevada de usuarios simultáneos.

Por otra parte, el sistema no contempla funcionalidades avanzadas como la integración con plataformas institucionales, el envío automático de notificaciones o la administración de documentos digitales asociados con las solicitudes.

Estas limitaciones corresponden al alcance establecido para el proyecto y representan oportunidades para futuras ampliaciones.

Consideraciones finales

La implementación de BecaGest permitió desarrollar una solución modular que integra diferentes procesos relacionados con la gestión de becas estudiantiles mediante una arquitectura REST.

El proyecto reúne componentes de persistencia de datos, autenticación, autorización, validaciones, consultas y pruebas, demostrando la aplicación práctica de diferentes conocimientos adquiridos durante la formación académica.

La utilización de tecnologías como Django, Django REST Framework y MySQL permitió establecer una estructura funcional y organizada para el procesamiento de información.

En conjunto, BecaGest constituye una solución académica que demuestra la integración de herramientas de desarrollo backend y proporciona una base para la incorporación de nuevas funcionalidades y mejoras en futuras etapas.

Referencias bibliográficas

Django Software Foundation. (s. f.-a). *Django documentation (version 6.1).* https://docs.djangoproject.com/en/6.1/

Django Software Foundation. (s. f.-b). *Writing and running tests.* https://docs.djangoproject.com/en/6.1/topics/testing/overview/

Encode OSS Ltd. (s. f.-a). *Authentication.* Django REST framework. https://www.django-rest-framework.org/api-guide/authentication/

Encode OSS Ltd. (s. f.-b). *Django REST framework.* https://www.django-rest-framework.org/

Oracle. (2026). *MySQL 8.4 reference manual.* https://dev.mysql.com/doc/refman/8.4/en/

Postman. (s. f.). *Send API requests and get response data in Postman.* https://learning.postman.com/docs/use/send-requests/requests

Python Software Foundation. (s. f.). *Python 3.14 documentation.* https://docs.python.org/3.14/